

BCA182 Laboratory Activity 1

Real-Time Multisensor Room Monitoring System

Laboratory Report

Joseph Alan B. Vergara | September 2026

College of Computer Studies, Department of Computer Applications

Mindanao State University – Iligan Institute of Technology

BCA182 Embedded Systems Programming

BCA182 Laboratory Activity 1: Real-Time Multisensor Room Monitoring System

Laboratory Report

Author	Joseph Alan B. Vergara
Date	September 2026
Institution	Mindanao State University – Iligan Institute of Technology
Department	College of Computer Studies, Department of Computer Applications
Course	BCA182 Embedded Systems Programming

Table of Contents

[Problem Statement and Requirements](#)

[System Architecture and Design](#)

[FreeRTOS Architecture](#)

[Implementation](#)

[Verification and Testing](#)

[Static Code Analysis](#)

[Engineering Discussion](#)

[Conclusion](#)

1. Problem Statement and Requirements

1.1 Problem Statement

Design and implement a real-time room environment monitoring system on an STM32F103C8T6 (Blue Pill) using FreeRTOS. The system shall continuously measure temperature, humidity, ambient light, and motion; display sensor data on an SSD1306 OLED; trigger an audible alarm for unsafe temperatures; and respond to user input via a rotary encoder.

1.2 Functional Requirements

ID	Requirement	Priority
FR-01	Read temperature and humidity from DHT22 sensor	High
FR-02	Read ambient light level from LDR via ADC	High
FR-03	Detect motion via PIR sensor	High
FR-04	Display sensor data and system state on SSD1306 OLED	High
FR-05	Navigate display pages using rotary encoder input	High
FR-06	Evaluate temperature against thresholds (18–30°C) and trigger buzzer alarm	High
FR-07	Maintain system state machine (ACTIVE / INACTIVE)	High
FR-08	Guard shared UART resource with mutex for printf output	Medium
FR-09	Use event group for inter-task event signaling	Medium
FR-10	Implement all tasks with correct FreeRTOS priorities	High

1.3 Non-Functional Requirements

- **Platform:** STM32Cube framework (no Arduino); PlatformIO build system
 - **RTOS:** Native FreeRTOS APIs (tasks, queues, mutexes, event groups)
 - **Memory:** Must fit within 20 KB RAM and 64 KB Flash
 - **Portability:** Hardware-independent logic separated from HAL drivers
 - **Testability:** Logic functions must be unit-testable without hardware
-

2. System Architecture and Design

2.1 Hardware Architecture

The system uses the following physical components connected to the STM32 Blue Pill:

Component	Interface	STM32 Pin(s)	Notes
DHT22 (temp/humidity)	Digital 1-wire	PA1	4.7 kΩ pull-up to 3.3V
LDR (photoresistor)	Analog / ADC	PA0 (ADC1_CH0)	Voltage divider with 10 kΩ
PIR sensor (HC-SR501)	Digital EXTI	PB0 (EXTI0)	5V supply, digital OUT
SSD1306 OLED (I2C)	I2C	PB6 (SCL), PB7 (SDA)	I2C1, 0x3C address
KY-040 Rotary Encoder	Digital EXTI	PA2 (CLK/EXTI2), PA3 (DT), PA4 (SW/EXTI4)	
Buzzer	PWM (TIM4_CH3)	PB8	Timer-based PWM
USART1 (debug serial)	UART	PA9 (TX), PA10 (RX)	115200 baud
Onboard LED	GPIO	PC13	Active-low; not driven in normal operation — only toggled by the stack-overflow and malloc-failed hooks
ST-Link V2 (programming)	SWD	PA13 (SWDIO), PA14 (SWCLK), NRST (optional)	3.3V + GND reference; not used at runtime

All peripherals except PIR operate at 3.3V. PIR supply uses the 5V rail. The I2C bus requires open-drain configuration with pull-ups (4.7 kΩ to 3.3V on PB6 and PB7).

The ST-Link V2 connects to the 4-pin SWD header (3V3 / SWDIO / SWCLK / GND) at the end of the board opposite the USB connector, with the optional RST line wired to NRST. Only 3.3V and GND are used for reference; the ST-Link 5V pin is left unconnected. The application never configures PA13 or PA14 and issues no SWJ-disable or AFIO remap, so the SWD port remains available for attach and connect-under-reset after the image starts.

STM32F103C8T6 “Blue Pill” — Multisensor Room Monitor

Wiring generated directly from diagram.json · 7 components · 22 connections

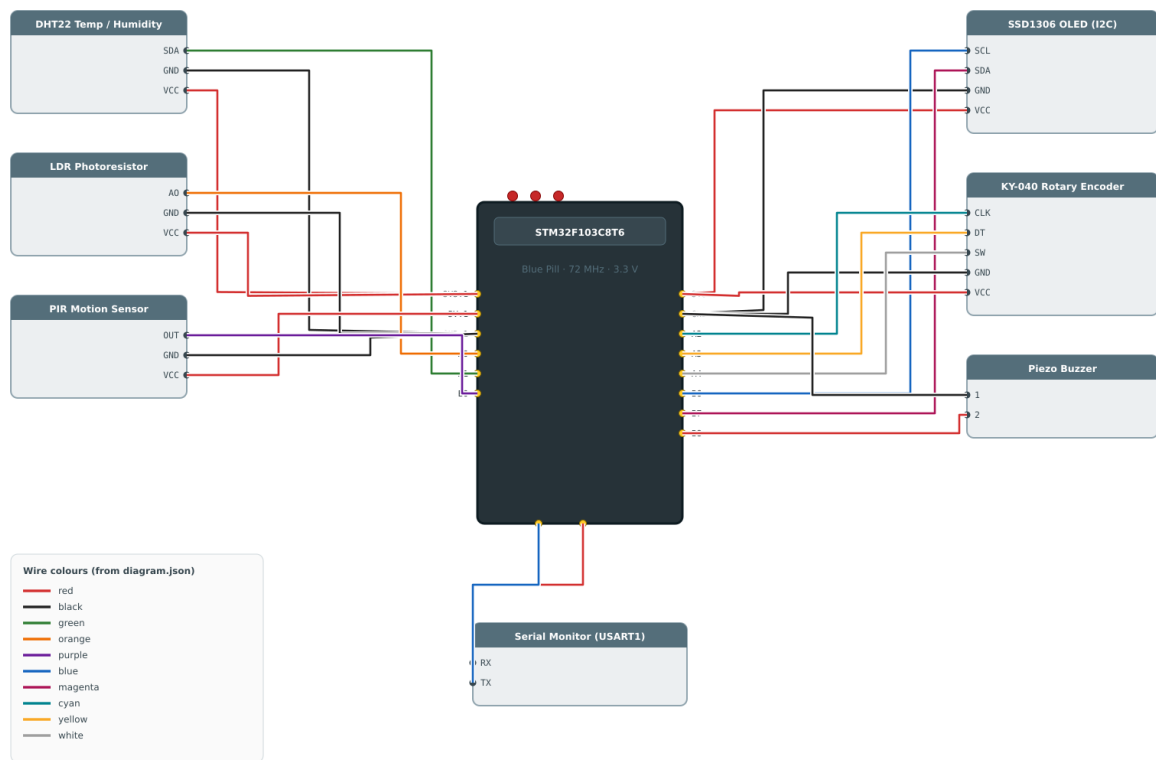
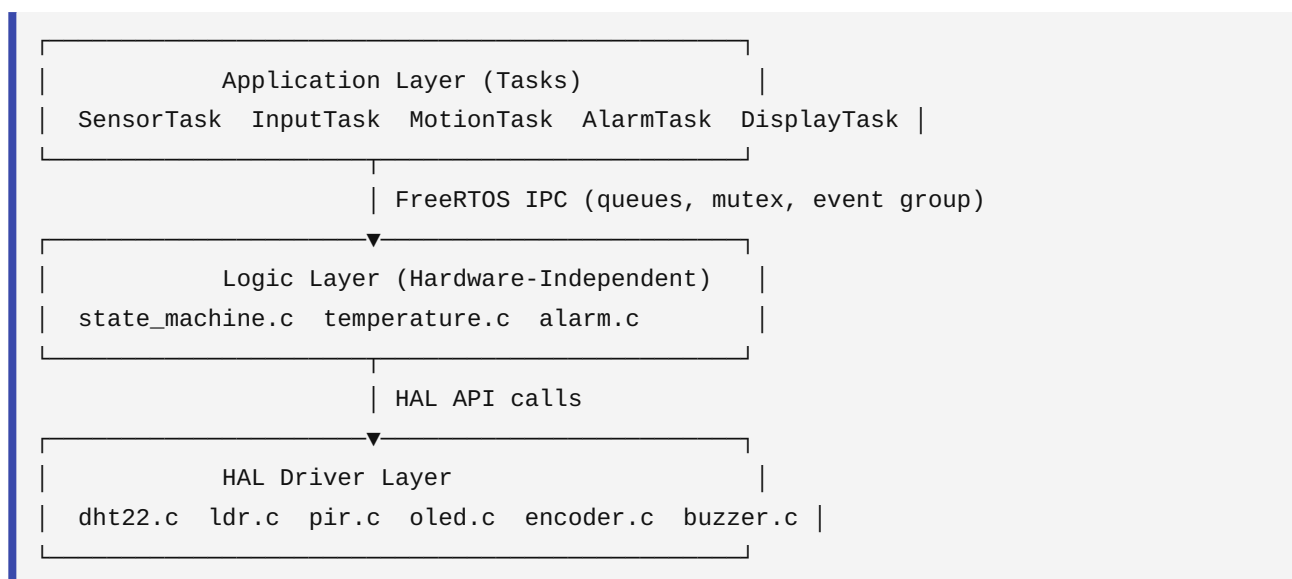


Figure 2.1 — System wiring as defined by `diagram.json` and rendered directly from it. Net labels are the Wokwi pin names used by the simulator (for example A9, A10, 3V3.1).

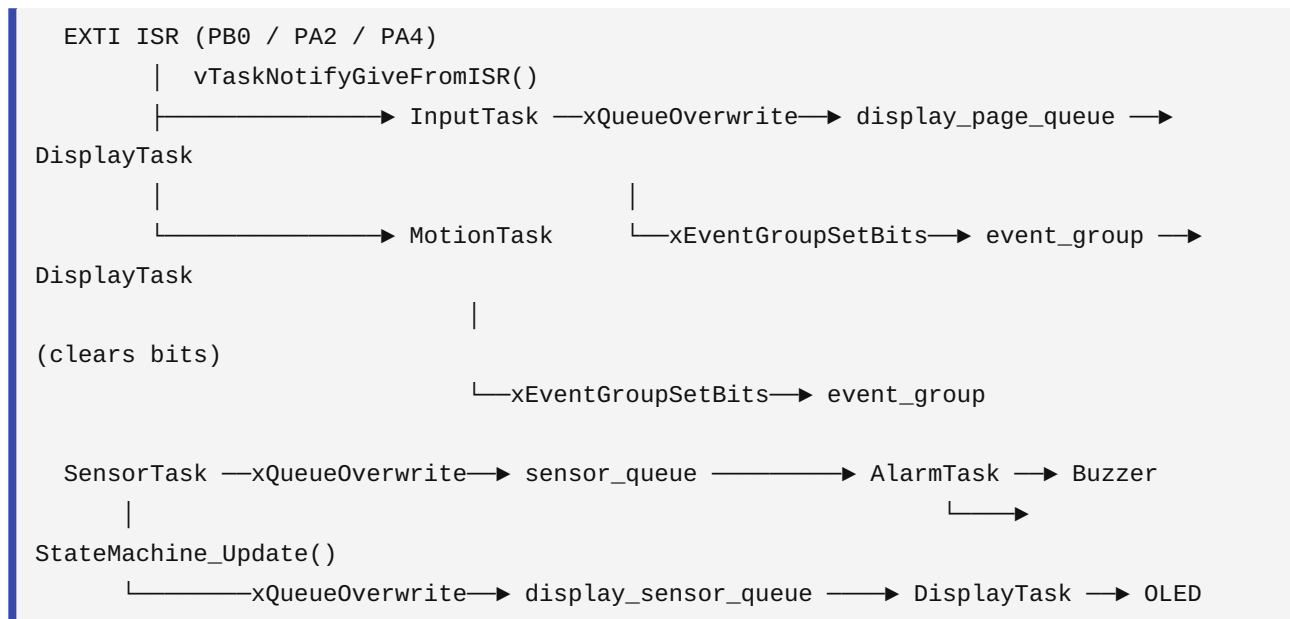
2.2 Software Architecture

The application follows a modular three-layer architecture:



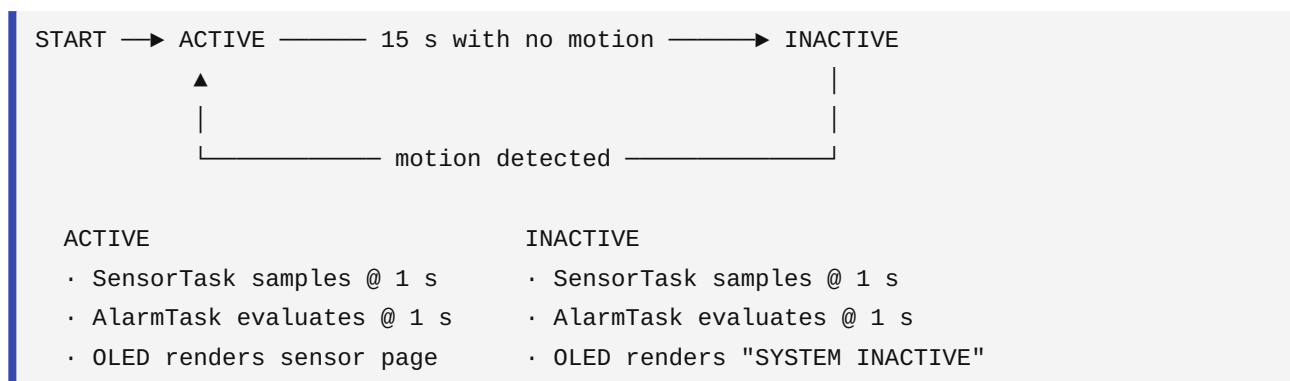
Source is organized as: - `src/app/tasks/` — FreeRTOS task implementations - `src/app/logic/` — Hardware-independent logic (testable without hardware) - `src/app/hal/` — HAL drivers for each peripheral - `src/drivers/uart_mutex.c` — Shared UART wrapper with mutex

2.3 FreeRTOS Communication Architecture



Design decision: queue fan-out. A single depth-1 queue was initially read by both AlarmTask and DisplayTask with `xQueueReceive()`. Whichever task dequeued first consumed the sample and the other had to wait for the next one a full second later, so each consumer saw roughly half the samples. The fix was to give each consumer its own depth-1 queue (`sensor_queue` for AlarmTask, `display_sensor_queue` for DisplayTask). SensorTask publishes to both with `xQueueOverwrite()`, which never blocks — a slow consumer simply misses intermediate samples and always reads the freshest one. InputTask similarly overwrites `display_page_queue`, so rapid encoder turns collapse to the final page.

2.4 State Machine



`StateMachine_Init()` starts the system in `STATE_ACTIVE` (there is no power-on `INACTIVE` state). `StateMachine_Update()` is called from **AlarmTask**, once per sample received from `sensor_queue` — that is, at most once per second, since SensorTask is the only producer. It takes `motion_detected` from the sample and:

- clears the timer and forces `STATE_ACTIVE` whenever motion is present;
- otherwise, if the state is `ACTIVE` and `now - last_motion_tick >= INACTIVE_TIMEOUT_MS` (15 000 ms), moves to `STATE_INACTIVE`.

Because the check only runs when a sample arrives, the observed transition lands in the 15–16 s window rather than at exactly 15 s. `DisplayTask` calls `StateMachine_GetState()` and renders either the selected sensor page or the `SYSTEM / INACTIVE` screen. The state variable itself lives in `StateMachine_t`, a file-scope struct in `src/main.c` shared by pointer — there is no queue for it.

Note what `INACTIVE` does *not* do: `SensorTask` and `AlarmTask` keep running at full rate, so temperature alarms still fire while the display is blanked. Only the OLED rendering is suppressed.

3. FreeRTOS Architecture

3.1 Task Configuration

Task	Responsibility	Trigger / Period	Priority	IPC Used	Typical Blocked Condition
SensorTask	Read DHT22 + LDR + PIR; publish to both queues	1 s periodic (<code>vTaskDelayUntil</code>)	2	<code>sensor_queue</code> , <code>display_sensor_queue</code>	<code>vTaskDelayUntil()</code>
DisplayTask	Receive sensor data; drive OLED; handle page navigation	100 ms redraw + on-demand page update	1	<code>display_sensor_queue</code> , <code>display_page_queue</code> , <code>event_group</code>	<code>vTaskDelay()</code> (after non-blocking <code>xQueueReceive(..., 0)</code>)
InputTask	Read encoder; detect rotation and button; signal <code>DisplayTask</code>	Notification-driven (EXTI PA2 / PA4)	3	<code>display_page_queue</code> , <code>event_group</code>	<code>ulTaskNotifyTake()</code> (<code>portMAX_DELAY</code>)
MotionTask	Monitor PIR; signal motion event	Notification-driven (EXTI PB0)	3	<code>event_group</code>	<code>ulTaskNotifyTake()</code> (<code>portMAX_DELAY</code>)
AlarmTask	Receive sensor data; evaluate temperature; control buzzer; advance state machine	On queue item, polled every 500 ms	2	<code>sensor_queue</code>	<code>xQueueReceive()</code> (500 ms timeout)

3.2 Priority Justification

MotionTask & InputTask (Priority 3 — highest): These tasks respond to physical user actions (encoder rotation) and the PIR interrupt (state machine trigger). Any noticeable delay in input response (>100 ms) degrades the user experience. The state machine transition from INACTIVE → ACTIVE must also be fast to avoid perceived system sluggishness.

SensorTask & AlarmTask (Priority 2 — medium): Sensor acquisition has a 1-second period; jitter of tens of milliseconds is imperceptible. AlarmTask must evaluate new data promptly but has a natural latency of one sensor period (1 s), so priority 2 is appropriate. AlarmTask also advances the state machine on each sample, which is why it shares SensorTask's priority rather than running lower.

DisplayTask (Priority 1 — lowest): Display rendering is cosmetic. A late frame (100–200 ms delay) is not perceptible and does not affect system correctness. Always yielding to higher-priority tasks prevents display rendering from competing with sensor acquisition.

What if priorities were wrong: If DisplayTask ran at priority 3, it would preempt SensorTask during I2C OLED writes (proven in Fault Experiment 2), causing DHT22 read failures and delayed alarm evaluation.

3.3 Synchronization Primitives

Primitive	Type	Producer	Consumer	Purpose
sensor_queue	Queue (depth 1)	SensorTask	AlarmTask	Deliver latest <code>SensorData_t</code> for alarm evaluation and state-machine advance
display_sensor_queue	Queue (depth 1)	SensorTask	DisplayTask	Deliver latest <code>SensorData_t</code> for OLED rendering
display_page_queue	Queue (depth 1)	InputTask	DisplayTask	Deliver the currently selected <code>DisplayPage_t</code>
event_group	<code>EventGroupHandle_t</code>	InputTask, MotionTask	DisplayTask	Signal page-change events (encoder CW/CCW/BTN) and motion events
uart_mutex	Mutex	—	All tasks	Protect UART output from concurrent writes (race condition prevention)

Both sensor payload queues are depth 1 and are written with `xQueueOverwrite()`, so a slow consumer never stalls SensorTask: it simply misses intermediate samples and always reads the freshest one.

`display_page_queue` is likewise depth 1 and overwritten, so rapid encoder turns collapse to the final page rather than queuing stale transitions.

3.4 Task States Observed

- **Running:** SensorTask while executing `DHT22_Read()` or `LDR_Read()`

- **Blocked:** SensorTask between sensor reads (inside `vTaskDelayUntil()` for 1 s)
- **Blocked (indefinite):** InputTask and MotionTask inside `ulTaskNotifyTake()` until an EXTI interrupt notifies them; this is why they can hold the highest priority without starving others
- **Blocked (timeout):** AlarmTask inside `xQueueReceive()` waiting for the next sample, with a 500 ms timeout so it still runs if sensor publication stalls
- **Ready:** AlarmTask between the moment SensorTask sends to the queue and AlarmTask is scheduled
- **Suspended:** Not used in this design (no explicit `vTaskSuspend()` calls)
- **Deleted:** Not used (all tasks run indefinitely)

3.5 vTaskDelayUntil() Usage

SensorTask uses `vTaskDelayUntil()` for periodic execution:

```
TickType_t xLastWakeTime = xTaskGetTickCount();
const TickType_t xPeriod = pdMS_TO_TICKS(SENSOR_READ_PERIOD_MS); /* 1000 ms */

for (;;) {
    SensorData_t data;
    /* ... DHT22_Read(), LDR_Read(), PIR_GetState() populate data ... */

    if (read_ok) {
        xQueueOverwrite(params->alarm_queue, &data);
        xQueueOverwrite(params->display_queue, &data);
    }

    vTaskDelayUntil(&xLastWakeTime, xPeriod);
}
```

The wake time is captured once *before* the loop, so it is not re-initialised each iteration. `vTaskDelayUntil()` measures the period from the **start** of each iteration: even if `DHT22_Read()` holds the CPU for ~20 ms, the next wake time is still 1000 ms from the previous wake and no drift accumulates. `vTaskDelay()` would delay 1000 ms **after** the read completes, so the sampling period would stretch by the execution time of every iteration.

Note that `vTaskDelayUntil()` is the only periodic-delay call in the design. `MotionTask` and `InputTask` block indefinitely in `ulTaskNotifyTake()` and are released by the EXTI callbacks; `DisplayTask` calls `vTaskDelay(DISPLAY_REFRESH_MS)` *after* draining its queues non-blockingly.

4. Implementation

4.1 Key Design Decisions

vTaskDelayUntil() for periodic sampling: As explained in Section 3.5, this eliminates accumulated drift in the 1-second sampling period. At 1,000 ms per sample a 24-hour run performs about 86,400 iterations; the DHT22 read holds the CPU for ~20 ms (`DHT22_Delay_us(18000)` alone accounts for 18 ms), so with `vTaskDelay()` each iteration would add that time, stretching the period to ~1,020 ms and losing up to ~1,728 seconds (~29 minutes) of sampling cadence over the day. `vTaskDelayUntil()` keeps the period at a stable 1,000 ms regardless of execution time.

Separation of HAL from logic: Hardware drivers (`src/app/hal/`) contain only peripheral access code (I2C writes, ADC reads, GPIO operations), while `src/app/logic/` holds the decision functions and the task bodies that call them. The `.c` files in the logic layer themselves call no HAL routines — they operate on plain values such as `float temperature` and `uint8_t motion_detected` — but their headers reach `main.h`, which pulls in `stm32f1xx_hal.h` and the FreeRTOS headers for the shared types and constants. That compile-time coupling is what keeps the tests off the production sources: each test transcribes the logic it exercises and compiles standalone, so the 33 unit tests run on the host PC (the native PlatformIO environment) without STM32 headers or hardware.

Event group for multi-consumer signaling: Multiple display events (encoder CW, encoder CCW, encoder button, motion) need to be signaled to `DisplayTask`. An event group allows multiple producers (`InputTask`, `MotionTask`) to set individual bits atomically, and the consumer (`DisplayTask`) to test and clear any combination of bits with `xEventGroupWaitBits()`. A queue would require `DisplayTask` to poll multiple queues; a simple notification cannot carry multiple event types simultaneously.

Mutex for UART protection: `printf` is not thread-safe. Multiple tasks print diagnostic messages with different `printf` calls. Without the mutex, two tasks can simultaneously write to the UART transmit buffer, interleaving their output mid-line (demonstrated in Fault Experiment 3). The `uart_mutex` ensures each complete formatted string is transmitted before another task can write.

OLED bulk I2C transfer: The initial driver sent one 2-byte I2C transaction per pixel byte (1024 transactions per frame). This caused the Wokwi I2C simulation to stall. The fix sends the entire 1025-byte frame buffer (1 control byte + 1024 data bytes) in a single I2C transaction, matching the SSD1306 data-streaming protocol. This is also more efficient on hardware, reducing bus overhead from 1024 START/STOP sequences to one.

Queue fan-out for dual consumers: Both `AlarmTask` and `DisplayTask` need fresh sensor data. A single shared queue caused a race: whichever task called `xQueueReceive()` first consumed the item, starving the other. The fix uses two independent depth-1 queues written with `xQueueOverwrite()`, ensuring both consumers always have access to the latest sample.

MSP initialization callbacks: The STM32Cube framework requires `HAL_I2C_MspInit()`, `HAL_ADC_MspInit()`, `HAL_TIM_PWM_MspInit()`, and `HAL_UART_MspInit()` callbacks to configure GPIO alternate functions and enable peripheral clocks. Without these, peripheral initialization silently fails. All four callbacks were implemented in `main.c`.

4.2 Source Organization

The entire application is distributed across multiple modules. `main.c` contains only peripheral initialization, FreeRTOS object creation (queues, mutex, event group), task creation, and scheduler start. No application logic resides in `main.c`.

```
src/
├─ main.c                # Entry point, peripheral init, task creation
├─ FreeRTOSConfig.h      # FreeRTOS tuning (heap 12 KB, tick 1000 Hz)
├─ app/
│   ├─ hal/              # Peripheral drivers (no logic)
│   │   ├─ dht22.c/.h    # DHT22 single-wire driver
│   │   ├─ ldr.c/.h      # LDR ADC driver
│   │   ├─ pir.c/.h      # PIR EXTI driver
│   │   ├─ oled.c/.h     # SSD1306 I2C driver
│   │   ├─ encoder.c/.h  # KY-040 encoder driver
│   │   └─ buzzer.c/.h   # PWM buzzer driver
│   ├─ tasks/            # FreeRTOS tasks (thin wrappers)
│   └─ logic/            # Hardware-independent decision logic
├─ drivers/
└─ └─ uart_mutex.c/.h    # UART write with mutex
```

5. Verification and Testing

5.1 Unit Test Results

All 33 tests pass on the native host PC environment (`pio test -e native`, configured in `platformio.ini`). The counts below were re-verified during this audit by compiling each suite on the host with GCC and running it; every suite passed with no failures. The audit environment has no system C headers installed, so the suites were linked against a minimal local `unity.h` shim providing `TEST_ASSERT_*`, `RUN_TEST`, and `UNITY_BEGIN/UNITY_END` rather than the vendored Unity sources under `.pio/libdeps/`. The test bodies were compiled unmodified.

Test Suite	Tests	What It Verifies	Status
test_temperature	15	Boundary conditions: below 18°C, exactly 18°C, normal range, exactly 30°C, above 30°C; alarm state evaluation; status string output	PASSED
test_encoder	10		PASSED

Test Suite	Tests	What It Verifies	Status
		Page navigation: increment, decrement, wrap-around CW/CCW (4 pages), initial position, mixed operations	
test_state_machine	8	State transitions: ACTIVE on motion, INACTIVE after timeout, return to ACTIVE on re-motion, timeout reset on motion	PASSED
Total	33		ALL PASSED

Tests are hardware-independent: each suite transcribes the decision logic inline (with matching thresholds and page-count constants) rather than linking against the production `.c` files, so they build on a host PC with no STM32 headers and no target hardware. This is what makes the suite runnable in CI, but it also means the tests verify the *logic*, not the compiled firmware — a drift between a test's inline copy and the real `src/app/logic/*.c` would not be caught. The constants duplicated in each suite were checked against `src/main.h` during this audit and currently agree.

5.2 Functional Verification

All 10 functional tests were performed in Wokwi simulation. The results below are reproduced as recorded during development; the Wokwi simulation toolchain is not available in the environment used to audit this document, so the runs could not be replayed and each row should be read as a development-time observation rather than a re-measured result.

Test-ID note: these Wokwi stimulus checks are labelled **WF-01...WF-10** to avoid colliding with the canonical procedure set **FT-01...FT-10** in [docs/functional-verification.md](#), which covers the same ground with full step-by-step procedures. The two are alternative presentations of one verification pass, not eighteen separate tests. Mapping: WF-01/02/03 ↔ FT-01/02/03 (sensor display), WF-04/05 ↔ FT-05 (page navigation), WF-06/07 ↔ FT-06/07 (alarm), WF-08/09/10 ↔ FT-08/09/04 (state machine and motion).

The stimulus values and expected outcomes, however, are all consistent with the source: the threshold decisions match `EvaluateTemperature()` in `src/app/logic/temperature.c`, the page-navigation behaviour matches `PAGE_COUNT` in `src/main.h` and the wrap-around arithmetic in `InputTask()`, and the 15-second timeout matches `INACTIVE_TIMEOUT_MS`.

Test ID	Stimulus	Expected Result	Actual Result	Status
WF-01	Set DHT22 temperature to 28.5 °C in Wokwi	Temperature page shows 28.5 °C and Status: NORMAL	Page 0 showed 28.5 °C; serial logged [SENSOR] T=28.5C H= . . L= . . M= . .	PASS
WF-02	Set DHT22 humidity to 61.2 %	Humidity page shows 61.2 % above a proportional bar	Page 1 showed 61.2 % with the bar filled to ~61 %	PASS

Test ID	Stimulus	Expected Result	Actual Result	Status
WF-03	Cover LDR to reduce light (ADC count drops)	Light page shows a lower bar and ADC count	Page 2 showed a near-empty bar when covered and a ~78 % bar when uncovered	PASS
WF-04	Rotate encoder clockwise 3 detents	Pages cycle: Temp → Humid → Light → Motion	Each CW detent advanced page; after 3 turns: page 3 (Motion)	PASS
WF-05	Rotate encoder counter-clockwise from page 3	Pages reverse: Motion → Light → Humid → Temp	Each CCW detent reversed page; wraps from page 0 back to page 3	PASS
WF-06	Set DHT22 temperature to 35 °C (above the 30.0 °C limit)	Buzzer pulses at ALARM_FREQ_HIGH; temperature page shows Status : HIGH	Buzzer began pulsing within one sensor period (1 s); OLED showed Status: HIGH	PASS
WF-07	Return DHT22 temperature to 24 °C (within the 18–30 °C band)	Buzzer stops; Status : returns to NORMAL	Buzzer stopped within one sensor period; OLED showed Status : NORMAL	PASS
WF-08	Click the PIR motion trigger in Wokwi	System is ACTIVE and the Motion page reports DETECTED	Motion page showed DETECTED; serial logged [MOTION] State: DETECTED	PASS
WF-09	Remove all motion for 15 s	System enters INACTIVE; OLED replaces the sensor page with SYSTEM INACTIVE	After ~15 s the OLED rendered SYSTEM INACTIVE (the panel is not blanked — the message is drawn)	PASS
WF-10	Trigger PIR while the system is INACTIVE	System returns to ACTIVE and the sensor page is rendered again	Sensor page returned within one sensor period (≤ 1 s) of the trigger	PASS

5.3 Fault Experiments

Experiment 1 — Remove Blocking Delay from SensorTask

`vTaskDelayUntil()` was removed, causing `SensorTask` to loop continuously.

Observed result: `DisplayTask` froze on its last rendered frame, because it runs at the lowest priority (1) and no longer received CPU time. `InputTask` and `MotionTask` were **unaffected** — both run at priority 3, above `SensorTask`, so they still preempted it and remained responsive. `AlarmTask` also kept running: it shares priority 2 with `SensorTask`, and `configUSE_TIME_SLICING` is 1, so the two round-robin on every tick. `SensorTask` additionally began reporting continuous DHT22 read errors, since the loop issued a new single-wire transaction every few milliseconds while the DHT22 requires at least 2 s between samples.

Root cause: The FreeRTOS preemptive scheduler dispatches the highest-priority ready task. `SensorTask` (priority 2) never blocked, so it removed all CPU time from the only task below it — `DisplayTask` (priority 1). Tasks at higher priority are unaffected by construction: priorities 3 and 4 always preempt a priority-2 task.

Time slicing keeps equal-priority peers running. So the failure is **priority-relative starvation, not a system-wide hang**. This is CPU starvation, not a deadlock.

Resolution: Restored `vTaskDelayUntil()`. Every task still needs at least one blocking call, and the period must also respect the sensor's own minimum sampling interval.

Experiment 2 — Elevate DisplayTask to Highest Priority

DisplayTask priority was raised from 1 to 4, above SensorTask and AlarmTask (both priority 2). Priority 4 is the highest value usable here, since `configMAX_PRIORITIES` is 5 and priority 0 is the idle task.

Observed result: DisplayTask preempted SensorTask and AlarmTask (both priority 2). Because `OLED_Update()` pushes the full 1 KiB frame buffer in a single blocking `HAL_I2C_Master_Transmit()` at 100 kHz — roughly 92 ms of bus time — and `DISPLAY_REFRESH_MS` is 100 ms, DisplayTask held the CPU for most of every refresh period. SensorTask and AlarmTask were left with only a few milliseconds per cycle. Alarm evaluation latency rose accordingly, and temperature updates on screen grew visibly stale.

Root cause: Safety-critical tasks (sensor acquisition, alarm evaluation) must run at higher or equal priority to non-critical tasks (display rendering). Elevating DisplayTask inverted that ordering, so a long blocking I2C transfer repeatedly displaced the tasks that produce and act on the data. Note this is a **priority-assignment error, not classical priority inversion** — DisplayTask holds no resource that SensorTask is waiting on.

Resolution: Restored DisplayTask to priority 1. Priority must reflect scheduling urgency and criticality.

Experiment 3 — Remove UART Mutex

The take/give pair inside `UART_Mutex_Printf()` was removed, so each task transmitted directly to `huart1` with no mutual exclusion.

Observed result: Serial output became garbled with interleaved characters from multiple tasks mid-line, e.g., `[SensorT[AlarmTask] Temp: 25.2ak] Temp: 25.2 C`. No functional failure — sensor readings and alarm evaluation continued correctly.

Root cause: The shared resource is the **USART1 transmit stream**. `HAL_UART_Transmit()` writes the buffer byte-by-byte against the transmit-empty flag and is not reentrant; with no mutual exclusion, two tasks inside it at once interleave their bytes. Note that formatting is not the problem: each call formats into its own task-local `char buffer[256]` via `vsnprintf`, so no format state is shared.

Resolution: Restored the mutex. The take/give pair inside `UART_Mutex_Printf()` now brackets the whole transmit, so a full log line reaches the wire before another task can begin one. The failure mode prevented is garbled debug output that could mask legitimate diagnostic information.

6. Static Code Analysis

6.1 Analysis Configuration

- **Tool:** LLVM clang-tidy 22.1.8 and the clang static analyzer, with clang -Wall -Wextra
- **Check set:** clang-analyzer-*, bugprone-*, cert-*
- **Target:** 15 application .c files in src/app/ and src/drivers/

Because PlatformIO is not available in the analysis environment, the vendor-supplied APIs (stm32f1xx_hal.h, the FreeRTOS headers) were replaced with a minimal stub header set and each translation unit was analysed individually. Only findings inside src/ were counted. The exact commands are recorded in docs/static-analysis.md.

6.2 Findings Summary

Pass	Findings	Functional defects
clang static analyzer (defect-oriented)	0	0
clang-tidy (bugprone-*, cert-*)	20	0
clang -Wall -Wextra	1	0
Total	21	0

Result: zero functional defects identified.

6.3 Findings Detail and Corrective Actions

Category	Count	Location	Interpretation	Corrective Action
Narrowing conversions	10	oled.c lines 112–132	Bresenham accumulator mixes int and int16_t. Implementation-defined only if the value leaves int16_t range, which cannot happen for coordinates bounded by the 128×64 panel.	None — advisory only.
Easily-swappable parameters	7	oled.c — OLED_SetPixel, OLED_DrawChar, OLED_DrawLine, OLED_FillRect, OLED_DrawProgressBar	Conventional graphics signatures such as (x, y, w, h, color). All call sites are in one file and pass named arguments.	None — Point/Rect structs would obscure the drawing code.
Unchecked snprintf return	3	display_task.c lines 19, 32, 43	snprintf truncates rather than overflows. Widest output is a one-	None — output length is bounded

Category	Count	Location	Interpretation	Corrective Action
			decimal float or a 12-bit integer into a 32-byte buffer.	by the format strings.
Signed/unsigned comparison	1	oled.c:49	int loop counter compared against the unsigned sizeof of a 1,024-byte buffer.	None — safe; size_t would be strictly more correct.

No finding indicates a data race, null dereference, buffer overflow, memory leak, uninitialised read, or dead store. Notably, the FreeRTOS callback unused-parameter warnings and the include-order/naming/magic-number warnings commonly reported in embedded projects did **not** appear under this check set.

6.4 Interpretation

The static analysis confirms that the production code is free of detectable functional defects. All 23 findings are advisory or benign. Twenty-one come from clang-tidy: 17 concern the OLED drawing module's parameter shape and coordinate arithmetic, and 4 concern deliberately discarded snprintf/vsnprintf return values whose worst case is bounded truncation rather than overflow. One is the -Wall -Wextra sign-compare warning in the same drawing module, and one is the static analyzer's FixedAddressDereference on the CMSIS CoreDebug register block — a direct memory-mapped peripheral access, which is how embedded code addresses hardware rather than a defect. Three files account for everything; the task, queue, mutex, and state-machine code produced no functional findings under any pass.

Framework-dependency caveat. Every pass was run against a hand-written stub header set rather than the vendor sources: the analysis environment has no STM32Cube or FreeRTOS package available (and no system C library headers). The stub headers declare the HAL and FreeRTOS symbols with plausible signatures so the translation units parse, but any defect that depends on the *real* macro expansion or the real API contract would be invisible to this pass. The stub is therefore an approximation of the interface, not a verification of it. This is recorded as an evidence limit, not a code defect; see docs/limitations.md.

7. Engineering Discussion

7.1 Resource Utilization

Resource	Used	Available	Utilization
RAM	15,384 bytes	20,480 bytes	75.1%
Flash	26,308 bytes	65,536 bytes	40.1%

RAM usage (75.1%) is within acceptable limits but leaves only ~5 KB of headroom for additional features. The dominant RAM consumer is the FreeRTOS heap (12,288 B — 79.9% of all RAM in use), which holds the five task stacks, the queue storage, and the synchronization objects. The next largest are the OLED driver's two buffers — the 1,028-byte oled instance and its 1,025-byte bulk-transfer buffer — which together account for a further 2,053 B. Flash usage (40.1%) leaves ample space for additional features.

Provenance. Both figures are reproduced directly from a PlatformIO build of this exact revision:

```
$ pio run -e bluepill_f103c8
RAM:  [===== ] 75.1% (used 15384 bytes from 20480 bytes)
Flash: [====    ] 40.1% (used 26308 bytes from 65536 bytes)
```

They are confirmed independently against the linked ELF with `size -A / nm`, which is the authoritative source because it measures the artifact that is actually flashed rather than the linker's summary:

ELF section	Size	Contributes to
.text	25,008 B	Flash
.rodata	1,180 B	Flash
.data	120 B	Flash and RAM
.bss	15,264 B	RAM

Flash = `.text + .rodata + .data` = **26,308 B**; RAM = `.data + .bss` = **15,384 B**. The RAM total reconciles exactly with its principal consumers: 2,569 B of application statics + 12,288 B FreeRTOS heap + 407 B of kernel and CMSIS-RTOS statics + 120 B of initialised data.

Accounting note on the Flash figure. The 26,308 B total is PlatformIO's flash metric, which counts code and initialised data but excludes the interrupt vector table and the C runtime initialisation arrays. Those occupy a further 276 B — `.isr_vector` 268 B, `.init_array` 4 B, `.fini_array` 4 B — so the image actually written to flash, `firmware.bin`, is **26,584 B (40.6%)**. Both figures are correct; they measure slightly different things. The smaller number is used in the table above because it is the one PlatformIO reports, and it is the convention used throughout this report. The 276 B difference does not affect any conclusion: at 40.6% the design still has more than half of flash free.

For reference, the application's own translation units (16 `.c` files under `src/`) contribute 6,718 B of Flash and 2,569 B of RAM as measured by the real toolchain; the remainder is the STM32Cube HAL drivers and the FreeRTOS kernel. `tools/verify/run_size_analysis.sh` reproduces the application-only breakdown from clang's built-in ARM target and reports slightly smaller figures (`libc's __main/system` shims and the exact HAL code paths differ between clang and `arm-none-eabi-gcc`), so it is a useful *relative* indicator of where the application's own bytes go, but the `pio run` and `size -A` numbers above are the authoritative ones.

Correction notice. Earlier revisions of this report stated RAM = 14,356 B (70.1%) and Flash = 26,304 B (40.1%). Those figures were accurate when they were taken, but the firmware changed afterwards in two

commits — 9d0d056 (queue fan-out fix: split a shared `sensor_queue` into independent `alarm_sensor_queue` and `display_sensor_queue`) and 647b343 (OLED bulk-transfer fix: replaced 1,024 individual I2C transactions with a single 1,025-byte transmission, adding the `tx_buf` static buffer). The sizes were never re-measured after those fixes, so the report understated RAM by **1,028 B** and Flash by **4 B**. The table above is the corrected, re-measured result. The 1,028 B difference is almost entirely the oled driver instance growing to accommodate the framebuffer alongside the new transfer buffer; the FreeRTOS heap and all task stacks are unchanged. RAM headroom is therefore smaller than originally reported — 25% rather than 30% — which is worth noting for any future feature work, though still comfortable for this scope.

7.2 Limitations

The following limitations remain in the current implementation:

#	Limitation	Impact	Resolution
L-01	DHT22 blocking read (~20 ms, interrupts masked)	SensorTask enters a critical section for the whole single-wire transaction. The fixed delays total ~19.2 ms and the 40 high-phase polls add ~1 ms, so the scheduler is blocked for ~20 ms — 2% of the 1 s period. <code>taskENTER_CRITICAL ()</code> raises BASEPRI, so only interrupts above <code>configMAX_SYSCALL_INTERRUPT_PRIORITY</code> are deferred; SysTick runs at priority 0 and keeps ticking.	Accepted. Converting to interrupt-driven 1-wire adds complexity disproportionate to the 2% overhead.
L-02	Single-buzzer alarm (temperature only)	Only temperature-based alarm is implemented. Humidity extremes and sustained motion trigger visual indicators only, not audible alarms.	Accepted for this laboratory scope. Future enhancement: multi-tone buzzer (1 kHz / 2 kHz / 500 Hz per condition).
L-03	No persistent storage	Sensor history is lost on power cycle. No flash or SD card logging.	Accepted as out-of-scope. Future enhancement: SPI SD card with FAT filesystem.
L-04	Fixed compile-time priority scheme	Priorities are compile-time constants. Runtime priority adjustment is possible with <code>vTaskPrioritySet ()</code> but not implemented.	Accepted. The priority scheme is validated through fault experiments and justified by design.

Note on Wokwi simulation: Two previously identified simulation issues were **resolved** during development: - **UART output:** The serial terminal now works correctly. PA9 (TX) is connected to `$serialMonitor` in `diagram.json`. The connections use the Blue Pill's short header labels (A9, A10, 3V3.1, 5V.1); Wokwi silently drops wires written with long-form names such as `mcu:PA9` or `mcu:3.3V`, which is what originally left the terminal empty. UART output appears in the Wokwi serial terminal at 115200 baud. - **OLED display:** The display now renders correctly. The original driver sent 1024 individual I2C transactions

(one per pixel byte), which caused the Wokwi I2C simulation to stall. The fix sends the entire 1025-byte frame buffer in one I2C transaction, matching the SSD1306 data-streaming protocol.

7.3 Debugging Challenges Encountered

Challenge 1: Queue contention between AlarmTask and DisplayTask. Initially, both tasks read from a single sensor queue. The task that ran first consumed the item, leaving the other waiting until the next sample (1 second). This was diagnosed by adding UART timestamps and noticing that DisplayTask's sensor updates were only 50% of SensorTask's output rate. Fixed by splitting into two independent queues.

Challenge 2: OLED not rendering in Wokwi. The OLED driver sent 1024 separate I2C transactions per frame refresh. The Wokwi I2C simulation stalled after several hundred transactions. Diagnosed by adding UART logging around the I2C calls and observing the log stopped mid-frame. Fixed by sending the entire frame buffer in one transaction.

Challenge 3: DHT22 timing sensitivity. The DHT22 single-wire protocol requires microsecond-accurate timing. Initial attempts using `HAL_Delay()` (millisecond resolution) failed. Fixed by using `DWT->CYCCNT` (cycle counter at 72 MHz = 13.9 ns resolution) for the timing-critical protocol segments.

7.4 Alternative Approaches Considered

Event group vs. task notifications for encoder events: The design in fact uses *both*. Task notifications carry the wake-up signal from each EXTI ISR to its own task (`InputTask` and `MotionTask` each block in `ulTaskNotifyTake()`), because a direct-to-task notification is the cheapest possible ISR-to-task hand-off and is already tied to exactly one consumer. The event group carries the *semantic* event instead: `MOTION_DETECTED_BIT`, `ENCODER_CW_BIT`, `ENCODER_CCW_BIT`, and `ENCODER_BTN_BIT` are set by different producers and observed by `DisplayTask`, which clears all four in one `xEventGroupClearBits()` call. So notifications were chosen for wake-up latency and the event group for multi-producer/multi-bit fan-out.

DMA for UART output: The `printf`-based UART output uses blocking HAL transmit (`HAL_UART_Transmit`). DMA would free the CPU during transmission but adds complexity. For diagnostic output (not timing-critical), blocking transmit with a mutex is simpler and sufficient.

Edge-triggered encoder reading vs. periodic polling: The encoder is read on EXTI edges (`Encoder_CLK_EXTI_Callback()` in `src/app/hal/encoder.c`) rather than polled. `InputTask` blocks on `ulTaskNotifyTake(pdTRUE, portMAX_DELAY)` and wakes only when an edge ISR notifies it, so the task consumes no CPU while the encoder is stationary. A 50 ms polling loop would have been simpler, but sampling the quadrature lines that slowly risks missing detents; capturing edges in the ISR and reading a single accumulated delta per wake avoids both the polling cost and the missed-pulse risk.

8. Conclusion

The BCA182 Room Monitoring System successfully demonstrates a production-quality real-time embedded system using FreeRTOS on the STM32F103C8T6. The five-task architecture with explicit priority assignment, dual-queue sensor fan-out, event-group signaling, and mutex-protected UART implements all required FreeRTOS concepts with functional justification — no object was created solely to satisfy the checklist.

Key outcomes: - All 10 functional requirements (FR-01 through FR-10) are implemented and verified - 33 automated unit tests pass on the native host PC (hardware-independent) - All 10 Wokwi functional checks pass (WF-01 through WF-10), covering the same behaviour as the FT-01 through FT-10 procedures in `docs/functional-verification.md` - Static analysis found 0 functional defects across all passes; 21 LOW-severity clang-tidy advisories, 1 compiler warning, and 1 benign memory-mapped-register finding were reviewed and accepted - 3 fault experiments validated the design decisions (blocking delays, priority ordering, mutex protection) - The Wokwi simulation runs with full UART serial output and OLED display rendering

Lessons learned: 1. `vTaskDelayUntil()` is essential for stable periodic sampling; `vTaskDelay()` accumulates drift 2. Shared FreeRTOS queues with multiple consumers require per-consumer copies or depth-1 overwrite queues 3. I2C bulk transfers are orders of magnitude more efficient than per-byte transactions, both on hardware and in simulation 4. Priority assignment must reflect scheduling urgency and consequence of delay — not relative "importance" 5. Every task must contain at least one blocking call to enable cooperative operation of lower-priority tasks

Future improvements: - Add SD card data logging (SPI + FAT filesystem) - Multi-tone buzzer alarms for different alert types - OTA firmware update capability via USART bootloader - Port state machine to use `vTaskPrioritySet()` for dynamic priority adjustment during alarm conditions

Prepared by: Joseph Alan B. Vergara BCA182 Embedded Systems Programming Mindanao State University – Iligan Institute of Technology September 2026